

TASK-2 NEEDED TO BE COMPLETED :

Team Kickoff: Milestone 2 Architecture & Execution Plan

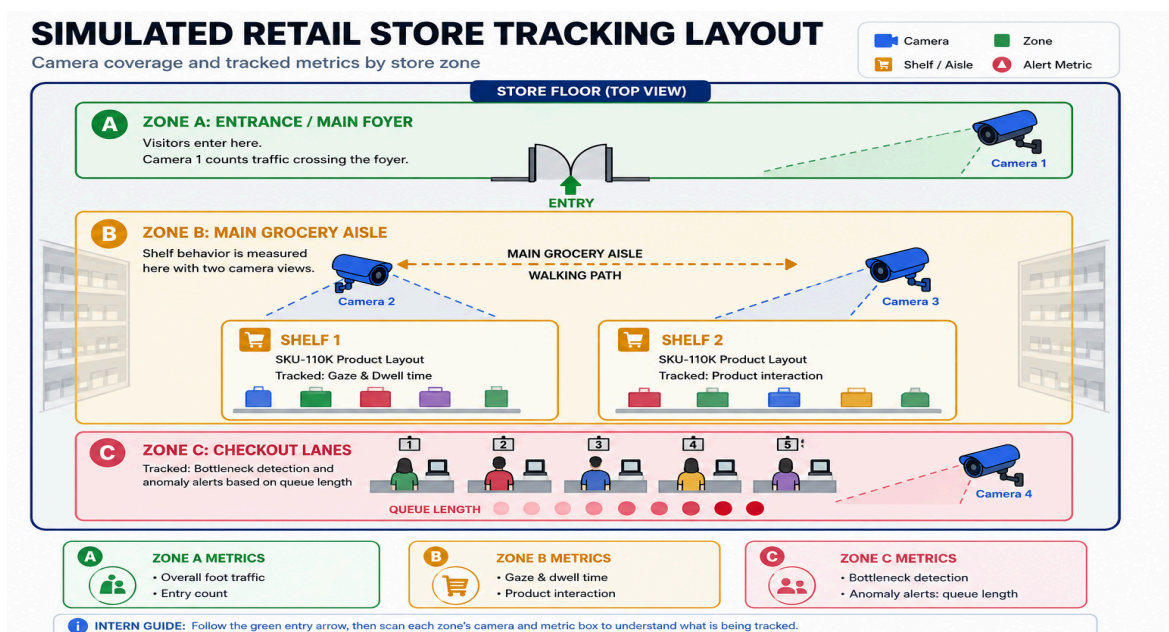
Hey Team,

Now that we have successfully wrapped up Milestone 1 (Infrastructure, Database schemas, and Auth), we are moving into Milestone 2 (Weeks 3 & 4): Core Processing, Tracking, and Intelligent Logic.

This is where we build the "AI brain" of the Consumer Attention Mapping System. Here is exactly how we are going to break this down, the tech stack we expect you to use, and the scope of the project.

1. The Scope: How many stores are we building for?

- **The Architecture:** Your backend and database **must be multi-tenant**. Use a `store_id` on your schemas. Do not hardcode a single store anywhere in your endpoints or database rows.
- **The Simulation:** For our actual testing and final demo, we are focusing on **one flagship virtual store**. Inside this single store, you will configure **3 distinct zones** using our video datasets:
 - **Zone 1: Entrance/Exit Foyer** (Monitored by Camera 1) – Used for counting net unique foot traffic.
 - **Zone 2: Main Product Aisle** (Monitored by Cameras 2 & 3) – Used for high-density shelf monitoring, gaze tracking, and product interaction.
 - **Zone 3: Checkout Lanes** (Monitored by Camera 4) – Used for bottleneck analysis and anomaly alert testing.



2. Dataset Assignment & Training Schedule

We are using the 4 datasets we initialized in Milestone 1. Here is exactly what each dataset is responsible for:

- **COCO Dataset:** Use this to extract baseline weights for tracking human bodies (Shoppers).
- **SKU-110K & Retail Product Checkout Datasets:** Use these to train the model to recognize tightly packed items on shelves and detect when a shopper interacts with a product.
- **Retail Store Traffic Dataset:** Use this pre-recorded data to simulate our live camera feeds and validate our analytical math.

 **Timeline Guardrail:** **Week 3** is strictly for data preprocessing, pipeline creation, and baseline object detection/tracking. **Week 4** is dedicated to Gaze estimation math, backend data routing, and building the frontend dashboard analytics.

3. Step-by-Step Technical Execution & Tech Stack

We expect you to follow this explicit pipeline. Do not deviate from these tools without checking in first:

Step-by-Step Technical Execution Plan:

Step 1: Data Pipeline Foundation & Preprocessing (Early Week 3)

- **The Task:** Prepare raw video files so the AI can read them, resize them, and process them smoothly without running out of computer memory.
- **The Dataset Application:** Built using raw video clips from the **Retail Store Traffic Dataset**.
- **How to Build It:**
 1. Use a video library to open the video file and chop it up into individual images (called frames) one by one.
 2. Resize those images so they match the exact size the AI model expects, and apply "data augmentation" (like adjusting brightness or flipping images) to help train a stronger model.
 3. Stream these images directly into the computer's graphics card (GPU) memory in smooth, organized batches instead of loading everything at once and crashing the system.
- **The Goal:** Build a reliable conveyor belt that feeds clean video frames to the AI as fast as possible.

Why?

An AI can't just watch a standard `.mp4` video file like a human does. It needs the video broken down into a rapid-fire sequence of individual pictures. Step 1 is all about building the "conveyor belt." If our conveyor belt is too slow, or if it tries to dump an entire 2 GB video into the computer's memory all at once, the system will instantly crash. We use specialized data pipelines to load just a few frames at a time, clean them up, and pass them to the AI's brain.

Step 2: Object Detection Model Training (Mid Week 3)

- **The Task:** Train the AI model's "eyes" so it can instantly spot both shoppers and retail products on shelves at the exact same time.
- **The Dataset Application:** Starts with a pre-trained human detector (COCO Dataset), then gets custom-trained using the SKU-110K and Retail Product Checkout datasets to find tiny items packed on store shelves.
- **How to Build It:**
 1. Set up the AI model architecture. We "freeze" the core parts of its brain that already know what basic shapes and humans look like.
 2. Perform "transfer learning" by showing it thousands of pictures of retail stores so it learns to recognize specific store layouts, shelves, and products.
 3. Program the model to draw a tight 2D "bounding box" around every single shopper and product it sees in every frame.
- **The Goal:** Achieve high accuracy in spotting people and items, even when shelves are crowded or shoppers are standing close together.

Why?

Instead of teaching an AI from scratch (which takes months and millions of pictures), we use a shortcut called Transfer Learning. We take a model that is already smart enough to recognize a human body, "freeze" that knowledge, and then train its remaining focus on recognizing retail products. The output of this step is a "bounding box"—a set of pixel coordinates that tells the system exactly where an object is located in the picture.

Tools to Use:

Tool	Purpose
Ultralytics YOLOv8 or YOLOv10	<i>The core AI framework used to look at the images and instantly detect objects.</i>
PyTorch / TensorFlow	<i>The heavy-duty engine running under the hood to calculate the AI's math.</i>
Weights & Biases (W&B) / TensorBoard	<i>A visual dashboard used to track the AI's learning progress and watch its error rates drop over time</i>

Step 3: Multi-Object Tracking (MOT) Implementation (Late Week 3)

- **The Task:** Ensure the AI system remembers a shopper's identity as they move through the video frame, instead of forgetting them every time they move.
- **The Dataset Application:** Validated and tested using the **Retail Store Traffic Dataset**.
- **How to Build It:**
 1. Connect a tracking algorithm directly to your object detection output.
 2. When the AI first spots a person, the tracker assigns them a persistent identification number (like "Shopper #105").
 3. The tracking software uses a mathematical filter to predict where that shopper will step next based on how fast they are moving.
 4. If the shopper is temporarily blocked from view by another person or an aisle pillar, the tracker makes sure their unique ID tag stays attached to them as soon as they step back into view.
- **The Goal:** Keep a seamless, continuous identity on every customer so the system can map individual paths accurately.

Why?

In **Step 2**, we taught the AI how to see objects (Detection). But object detection is "forgetful"—the moment a shopper walks behind a display shelf or gets blocked by a shopping cart, the AI loses them. When they step back out, a basic detector would treat them like a completely new person and assign a new number.

Step 3 adds **Prediction and Memory**. By tracking bounding boxes across frames, the algorithm calculates the shopper's velocity (speed and direction). Even if the AI loses sight of "Shopper #105" for a split second, it guesses where they should be based on their last known movement, keeping the data clean and continuous.

Step 4: Real-Time Multi-Object Tracking (Early Week 4)

- **The Task:** Teach the AI to assign a unique, permanent ID number to every individual shopper who walks into the store, tracking their movement continuously without losing them.
- **The Dataset Application:** Built and tested using video files from the **Retail Store Traffic Dataset**.
- **How to Build It:**
 - First, use a detection model (**YOLOv8**) to draw a bounding box around every person it sees in a video frame.
 - Next, attach a tracking algorithm (**ByteTrack**) to those boxes. ByteTrack looks at where a person was in the *previous* frame and matches them to where they are in the *current* frame.
 - Even if a shopper briefly walks behind a pillar, turns around, or gets blocked by another person, the algorithm remembers their unique ID rather than treating them like a brand-new customer when they reappear.
- **The Goal:** Keep a precise, continuous count of shoppers and follow their paths through the store without mixing up their identities.

Why?

Think of **YOLOv8** as the "eyes"—it can spot a person instantly, but it has no memory. It resets every single frame. If someone blinks, YOLO forgets who they are.

To fix this, we add **ByteTrack/DeepSort**, which acts as the "brain." It remembers the history and direction of each bounding box. By pairing them together, we get a system that doesn't just see a customer; it tracks *Customer #14* all the way from the entrance to the checkout aisle.

Step 5: Decoupled Ingestion & Stream Processing (Mid Week 4)

- **The Task:** Prevent the application from freezing, lagging, or crashing when handling high volumes of real-time AI tracking data.
- **The Dataset Application:** Validated by feeding continuous data streams simulated from the **Retail Store Traffic Dataset**.
- **How to Build It:** Video data typically streams at 30 frames per second (FPS). When tracking multiple shoppers simultaneously, the AI model generates hundreds of individual location coordinates every second, creating a massive data ingestion bottleneck if written directly to a standard database.
- **The Goal:** Ensure the application can gracefully ingest and process high-throughput AI tracking data in real time without locking up or losing data frames.
- **Tools To be used:**

Tool	Purpose
Redis Streams	<i>Acts as the lightweight, ultra-fast, in-memory data "waiting room" queue to handle raw ingest.</i>
TimescaleDB	<i>A specialized PostgreSQL extension built to store and query high-speed, time-series coordinate logs efficiently.</i>
Redis (Key-Value/Hashes)	<i>Used for maintaining instant, real-time state data, such as tracking the current live store occupancy.</i>
FastAPI WebSockets	<i>Used to push live coordinates instantly to the frontend dashboard without database polling overhead.</i>

System Architecture Explanation

Because video runs at 30 FPS, tracking multiple people simultaneously means the AI generates hundreds of coordinate pairs every single second. Attempting to save these coordinates to a traditional database one by one will instantly overwhelm the system, causing database locks and application freezes.

To solve this, the architecture introduces a decoupled ingestion pipeline using a fast "waiting room" queue. The AI rapidly drops raw coordinates into this ultra-low-latency queue and immediately returns to processing the next video frame. A separate, asynchronous background worker process then collects those coordinates from the queue, aggregates them into neat batches of 100, and performs a bulk save into the permanent time-series database every few seconds. Concurrently, these coordinates are instantly broadcast to the live website dashboard using WebSockets.

Step 6: Frontend Heatmap & Analytical Dashboard (Late Week 4)

- **The Task:** Turn raw coordinate logs into colorful, interactive charts and maps for store management.
- **The Dataset Application:** Visualizes the processed tracking history of the **Retail Store Traffic Dataset**.
- **How to Build It:** Build an analytical dashboard. Place the 2D layout map of your store on an interactive digital canvas. Create a code listener that catches the real-time coordinates coming from your backend WebSocket. Program the map to draw color gradients based on traffic density: paths walked frequently or shelves stared at for a long duration will glow **Red** (Hot Zones), while ignored spaces fade to **Blue** (Cold Zones). Build supporting components to show total customer counts and show alert pop-ups if overcrowding occurs.
- **Exact Tools to Use:** [Heatmap.js](#) or [simpleheat](#) (JavaScript rendering engines for drawing data heatmaps on HTML5 canvases), [Recharts](#) or [Chart.js](#) (for rendering data analytics graphs), and [Next.js](#) paired with [Tailwind CSS](#) and [Shadcn/ui](#) (for structuring the UI layouts and user dashboards).

Milestone 2 Summary:

In **Milestone 2**, the interns transition from working with a static AI model to building a **high-speed, live data tracking system**. While Milestone 1 focused on basic object detection, Milestone 2 is all about handling data over **time, continuity, and scale**.

Deliverables & Key Outcomes

By the end of Milestone 2, your interns will deliver:

- **Continuous Multi-Object Tracking:** The AI successfully tags every shopper with a permanent, unique ID (e.g., "Shopper #47") that stays with them throughout the video, even if they are temporarily blocked from view.
- **A Crash-Proof Data Pipeline:** A decoupled architecture using **Redis Streams** to instantly buffer incoming data and **TimescaleDB** to save coordinate batches efficiently without freezing the database.
- **Live Web Streaming:** A functional real-time connection using **FastAPI** **WebSockets** to push coordinates instantly from the backend to the browser with zero delay.
- **An Interactive Heatmap Dashboard:** A clean frontend interface built with **Next.js** and **Heatmap.js** that translates coordinate data into a glowing visual traffic map (Red for busy spots, Blue for empty areas) for store managers.

AI-Powered Retail Customer Tracking Architecture

